

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply

8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand
9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply
10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply

Code of Conduct:

I Sandra Sudevan-192424422 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sandra Sudevan

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	Demonstrates strong logical reasoning and clear	Shows reasonable analysis with	Limited analysis; weak reasoning	No analytical reasoning demonstrated

	justification of solutions	some justification		
Code Quality	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.

Incorrect Query:

```
CREATE TABLE Student
(  
  StudentID INT,  
  StudentName VARCHAR(50)  
  Age INT,  
  Email VARCHAR(100)  
);
```

Errors:

- Missing comma after Student Name.
- No PRIMARY KEY.
- EMAIL should be UNIQUE.
- Age should have a CHECK constraint.

Correct Query:

```
CREATE TABLE Student
(  
  StudentID INT PRIMARY KEY,  
  StudentName VARCHAR(50),  
  Age INT CHECK (Age >= 18),  
  Email VARCHAR(100) UNIQUE  
);
```

2. Debug an SQL query using aggregate functions without GROUP BY

Incorrect Query:

```
SELECT Department, COUNT(EmployeeID)  
FROM Employee;
```

Error:

- Department is a non-aggregate column, so GROUP BY is required.

Correct Query:

```
SELECT Department, COUNT(EmployeeID) AS TotalEmployees
FROM Employee
GROUP BY Department;
```

3. Correct a faulty SELECT query with misspelled column names

Incorrect Query:

```
SELECT StudntID, StudntName
FROM Student;
```

Error:

- Column names are misspelled.

Correct Query:

```
SELECT StudentID, StudentName
FROM Student;
```

4. Fix an SQL query using JOIN with a missing or incorrect join condition

Incorrect Query:

```
SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
JOIN Customers;
```

Error:

- Missing ON clause.

Correct Query:

```
SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
JOIN Customers
ON Orders.CustomerID = Customers.CustomerID;
```

5. Correct a subquery that returns multiple values instead of a single value

Incorrect Query:

```
SELECT EmployeeName
```

```
FROM Employee
WHERE DepartmentID =
(
    SELECT DepartmentID
    FROM Department
    WHERE Location = 'Chennai'
);
```

Error:

- The subquery may return multiple DepartmentID values, causing an error with =.

Correct Query:

```
SELECT EmployeeName
FROM Employee
WHERE DepartmentID IN
(
    SELECT DepartmentID
    FROM Department
    WHERE Location = 'Chennai'
);
```

6. Debug a VIEW creation statement that contains invalid column references or syntax errors.

Incorrect Query

```
CREATE VIEW Student_View AS
SELECT StudentID, Student_Name, Dept
FROM Students;
```

Errors

- Student_Name column does not exist.
- Dept column should be Department.

Correct Query

```
CREATE VIEW Student_View AS
SELECT StudentID, StudentName, Department
FROM Students;
```

7. Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.

Incorrect Query

```
SELECT *  
FROM Employee  
WHERE Salary > 30000  
AND Salary > 20000  
AND Department = 'HR'  
AND Department = 'HR';
```

Errors

- Salary > 20000 is unnecessary because Salary > 30000 already satisfies it.
- Duplicate department condition.

Correct Query

```
SELECT *  
FROM Employee  
WHERE Salary > 30000  
AND Department = 'HR';
```

8. Identify violations of integrity constraints in a given table and correct them.

Incorrect Table

```
CREATE TABLE Employee  
(  
    EmpID INT,  
    Name VARCHAR(50),  
    Age INT,  
    Email VARCHAR(100)  
);
```

Problems

- No Primary Key.
- Email should be Unique.
- Age should have a CHECK constraint.

Correct Table

```
CREATE TABLE Employee
(
    EmpID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Age INT CHECK (Age >= 18),
    Email VARCHAR(100) UNIQUE
);
```

9. Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.

Incorrect Relation

StudentID	StudentName	Subjects
101	Rahul	DBMS, Java, Python
102	Anu	C, C++, Java

Problem:

The Subjects column contains multiple values, violating First Normal Form (1NF).

Correct Relation (1NF)

Student Table

StudentID	StudentName
101	Ragul
102	Anu

Student_Subject Table

StudentID	Subject
101	DBMS
101	Java

101	Python
102	C
102	C++
102	Java

10. Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.

Unnormalized Table

StudentID	StudentName	DeptID	DeptName
101	Rahul	D01	CSE
102	Anu	D01	CSE
103	Ravi	D02	ECE

Problem

- DeptName depends only on DeptID, causing redundancy and violating 3NF.

Normalized Tables (3NF)

Department Table

Department(
 DeptID PRIMARY KEY,
 DeptName
)

DeptID	DeptName
D01	CSE
D02	ECE

Student Table

Student(
 StudentID PRIMARY KEY,
 StudentName,

DeptID,
FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
)

StudentID	StudentName	DeptID
-----------	-------------	--------

101	Rahul	D01
-----	-------	-----

102	Anu	D01
-----	-----	-----

103	Ravi	D02
-----	------	-----